

PostgreSQL Blue/Green Upgrade — Step-by-Step

Overview

A blue/green upgrade runs two PostgreSQL instances in parallel. Data is continuously synced from the old instance (blue) to the new instance (green) using logical replication. Once green is confirmed in sync, traffic is cut over to green. Blue is then decommissioned.

Key advantage over in-place upgrade: Zero data loss, easy rollback — just point back to blue if anything goes wrong on green.

Setup used:

- Blue: existing EC2 instance (source, running current PostgreSQL version)
- Green: new EC2 instance (target, running new PostgreSQL version)
- Sync method: logical replication
- Infrastructure: Terraform on AWS (EC2, VPC, Security Groups, Secrets Manager)

Infrastructure

Terraform — Add the Green Instance

Add a second EC2 instance (`aws_instance.pg_green`) in `ec2_pg_cluster.tf` with PostgreSQL 18 pre-configured, `wal_level = logical`, and `md5` auth enabled:

```
resource "aws_instance" "pg_green" {
  ami                = data.aws_ami.amazon_linux.id
  instance_type      = var.instance_type
  subnet_id          = aws_subnet.public[0].id
  vpc_security_group_ids = [aws_security_group.pg.id]
  iam_instance_profile = aws_iam_instance_profile.pg.name
  key_name            = aws_key_pair.pg.key_name
  associate_public_ip_address = true

  user_data = <<-EOF
    #!/bin/bash
    dnf install -y postgresql18-server postgresql18
    postgresql-setup --initdb
    systemctl enable postgresql && systemctl start postgresql
    sed -i "s/#listen_addresses = 'localhost'/listen_addresses = '*'/"
/var/lib/pgsql/data/postgresql.conf
    sed -i "s/#wal_level = replica/wal_level = logical/" /var/lib/pgsql/data/postgresql.conf
    sed -i 's/ident/md5/g; s/scram-sha-256/md5/g' /var/lib/pgsql/data/pg_hba.conf
    echo "host all all 0.0.0.0/0 md5" >> /var/lib/pgsql/data/pg_hba.conf
    echo "host replication all 0.0.0.0/0 md5" >> /var/lib/pgsql/data/pg_hba.conf
    systemctl restart postgresql
  EOF

  tags = { Name = "${var.project_name}-green" }
}
```

Push to GitHub — the CI/CD pipeline creates the green instance automatically.

Step 1 — Prepare Blue (Publisher)

SSH into the blue instance.

Set a password for the postgres user (if not already done)

```
sudo -u postgres psql -c "ALTER USER postgres PASSWORD 'YourPassword';"
```

Verify wal_level is logical

```
sudo -u postgres psql -c "SHOW wal_level;"
```

If it shows `replica`, enable it and restart:

```
sudo -u postgres psql -c "ALTER SYSTEM SET wal_level = logical;"
sudo systemctl restart postgresql
```

Fix pg_hba.conf to allow replication connections

```
sudo grep "^host" /var/lib/pgsql/data/pg_hba.conf
# All entries must show md5. If not:
sudo sed -i 's/ident/md5/g; s/scram-sha-256/md5/g' /var/lib/pgsql/data/pg_hba.conf
sudo systemctl reload postgresql
```

Create publications for all user databases

```
# Repeat for each database to replicate
sudo -u postgres psql -d publisher_db -c "CREATE PUBLICATION blue_pub_orders FOR TABLE
orders;"
```

Step 2 — Prepare Green (Subscriber)

SSH into the green instance.

Set the same postgres password as blue

```
sudo -u postgres psql -c "ALTER USER postgres PASSWORD 'YourPassword';"
```

Create matching database and table schemas

```
sudo -u postgres psql -c "CREATE DATABASE publisher_db;"
```

```
sudo -u postgres psql -d publisher_db -c "  
CREATE TABLE orders (  
  id          SERIAL PRIMARY KEY,  
  customer    VARCHAR(100),  
  product     VARCHAR(100),  
  amount      NUMERIC(10,2),  
  created_at  TIMESTAMP DEFAULT now()  
);"
```

Repeat for every database and table being replicated from blue.

Step 3 — Create the Replication Slot on Blue

SSH into blue and create the slot manually (avoids hang on loopback):

```
sudo -u postgres psql -d publisher_db -c \  
"SELECT pg_create_logical_replication_slot('green_sub', 'pgoutput');"
```

Verify:

```
sudo -u postgres psql -c "SELECT slot_name, active FROM pg_replication_slots;"
```

Step 4 — Create Subscriptions on Green

SSH into green. Replace `<BLUE_IP>` with the blue instance's public IP.

```
set +H    # Disable bash history expansion (needed if password contains !)  
  
sudo -u postgres psql -d publisher_db -c "CREATE SUBSCRIPTION green_sub CONNECTION 'host=  
<BLUE_IP> port=5432 dbname=publisher_db user=postgres password=YourPassword' PUBLICATION  
blue_pub_orders WITH (create_slot = false, slot_name = 'green_sub', copy_data = true);"
```

Options explained:

Option	Value	Reason
create_slot	false	Slot was pre-created on blue in Step 3
slot_name	green_sub	References the slot created in Step 3
copy_data	true	Copy all existing rows from blue to green

Step 5 — Verify Sync

On green, check the subscription is active:

```
sudo -u postgres psql -d publisher_db -c "SELECT subname, subenabled FROM pg_subscription;"
```

Check replication lag on blue:

```
sudo -u postgres psql -c "SELECT slot_name, active,
pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), confirmed_flush_lsn)) AS lag FROM
pg_replication_slots;"
```

Lag should be near `0 bytes`. Insert a test row on blue and confirm it appears on green:

```
# On blue
sudo -u postgres psql -d publisher_db -c "INSERT INTO orders (customer, product, amount)
VALUES ('TestCutover', 'Item', 1.00);"

# On green
sudo -u postgres psql -d publisher_db -c "SELECT * FROM orders WHERE customer =
'TestCutover';"
```

Step 6 — Cutover

Once green is confirmed in sync:

1. **Pause writes to blue** (stop the application or put blue in read-only mode):

```
# On blue
sudo -u postgres psql -c "ALTER DATABASE publisher_db SET
default_transaction_read_only = on;"
```

2. **Wait for replication lag to reach zero:**

```
# On blue – run until lag = 0 bytes
sudo -u postgres psql -c "SELECT pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(),
confirmed_flush_lsn)) AS lag FROM pg_replication_slots WHERE slot_name =
'green_sub';"
```

3. **Drop the subscription on green:**

```
sudo -u postgres psql -d publisher_db -c "DROP SUBSCRIPTION green_sub;"
```

4. **Update connection strings** to point to green's IP address (pgAdmin4, application config, etc.)
5. **Verify green is serving traffic correctly.**

Step 7 — Decommission Blue

Once green is confirmed healthy:

```
# Remove the blue instance from Terraform
# In ec2_pg_cluster.tf, delete the aws_instance.pg resource block
```

```
# Then push to GitHub to trigger destroy of blue only
```

Or destroy via Terraform targeted destroy:

```
terraform destroy -target=aws_instance.pg
```

Rollback Plan

If green has issues after cutover, revert in minutes:

1. Re-enable writes on blue:

```
sudo -u postgres psql -c "ALTER DATABASE publisher_db SET
default_transaction_read_only = off;"
```

2. Point connection strings back to blue's IP.
3. Investigate green before retrying cutover.

Comparison: Option A vs Option B

	Option A (In-Place)	Option B (Blue/Green)
Downtime	Brief (minutes)	Near zero
Rollback	Manual, complex	Instant — point back to blue
Data risk	Higher	Lower
Cost	Same instance	Temporary double cost
Complexity	Lower	Higher
Recommended for	Dev/test, small DBs	Production, critical systems

Troubleshooting

Symptom	Cause	Fix
CREATE SUBSCRIPTION hangs	Slot creation on loopback	Create slot manually first (Step 3)
Replication lag not decreasing	Network/firewall blocking port 5432	Check security group allows 5432 from green to blue
copy_data takes long	Large dataset	Normal — monitor with pg_stat_subscription
Green missing tables	Schema not created before subscription	Create all tables on green before subscribing

ident authentication failed	pg_hba.conf not updated	Run sed fix for md5 on blue
--------------------------------	-------------------------	-----------------------------